University of Nottingham
UK | CHINA | MALAYSIA

COMP2001/2011: Artificial Intelligence Methods

Lecture 8 – Hyper-heuristics 2

Dr Warren G Jackson

The landscape so far…

- Previously we have looked at – manual/expert designs:
  - Components of heuristic search
  - Simple “low-level” heuristics (perturbative/hill-climbing)
  - Single-point and Population based Metaheuristics
  - Selection Hyper-heuristics

- Today’s Topics:
  - Graph-based HHs with an application to Examination Timetabling
  - Genetic Programming
  - A Generative Constructive Hyper-heuristic for Bin Packing Problems

Graph-based Hyper-heuristics: A Constructive Approach

- Graph colouring
- Examination timetabling
- A graph-based Hyper-heuristic for Exam Timetabling Problems

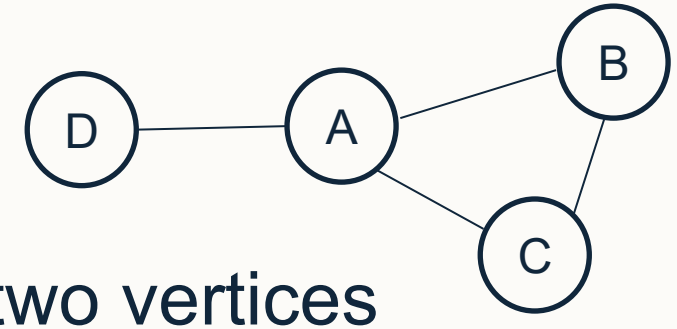
A Graph-based Hyper-heuristic

- Is a constructive approach.
- Low-level heuristics are all constructive.
- LLHs are simple graph colouring heuristics.
- The hyper-heuristic is trying to find the optimal sequence of constructive low-level heuristics to generate a complete solution.
- The completed coloured graph represents a solution to a timetabling problem.

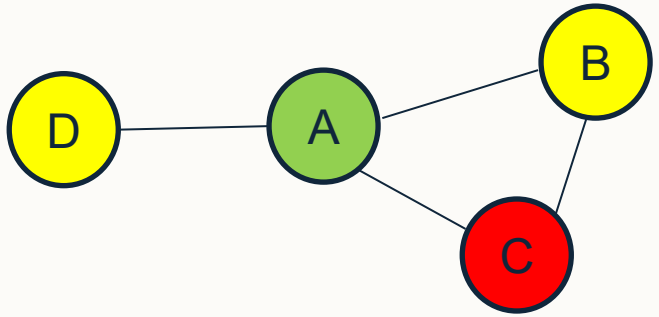
E. Burke, B. McCollum, A. Meisels, S. Petrovic, and R. Qu. A graph-based hyper-heuristic for educational timetabling problems. European Journal of Operational Research, 176(1):177-192

Graph Colouring

- Given a graph G containing a set of vertices V and edges $E \subseteq V \times V$



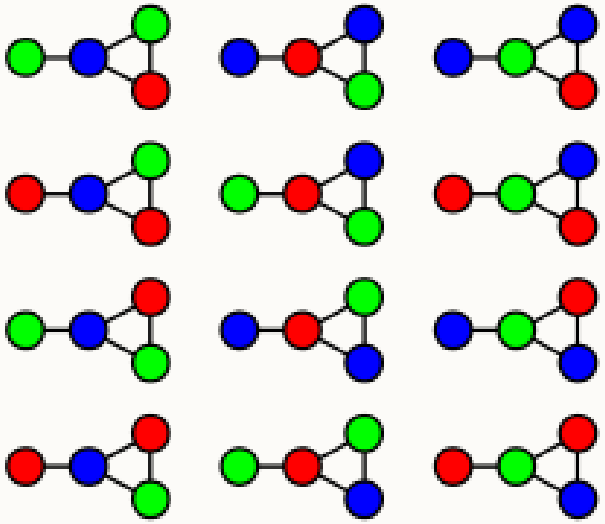
- Is it possible to colour each vertex such that no two vertices connected via an edge share the same colour? (Vertex colouring).
  - Vertices could represent exams.
  - Edges could be constraints caused by students taking multiple exams.
  - Trivial with $|V|$ colours but what if the number of exam slots is less than $|V|$?



Graph Colouring Problems

- **k-colouring problem**: can the vertices of the graph be coloured using k colours such that no two vertices with the same colour are connected via an edge?

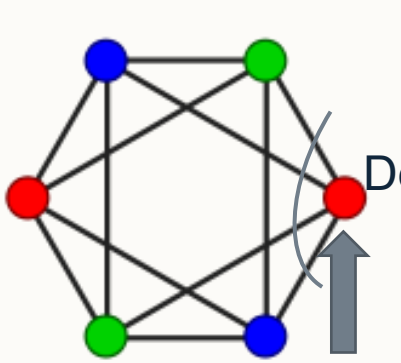
- **Minimum colouring problem**: Given a graph $G = (V, E)$ colour the vertices of the graph using the minimum number of different colours such that no two vertices with the same colour are connected via an edge. This is NP-Hard.



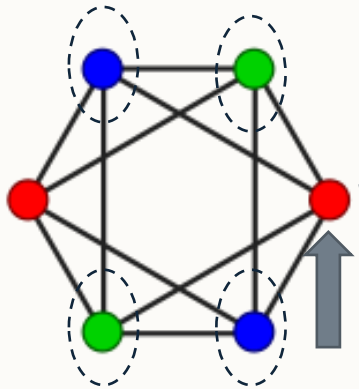
This graph can be 3-colored in 12 different ways.

Properties of Vertices

- The number of edges connected to a given vertex is called its **degree.**
- The number of differently coloured vertices connected to a given vertex is called its **saturation degree**.
- These properties can be used to create some constructive heuristics!



Degree: 4



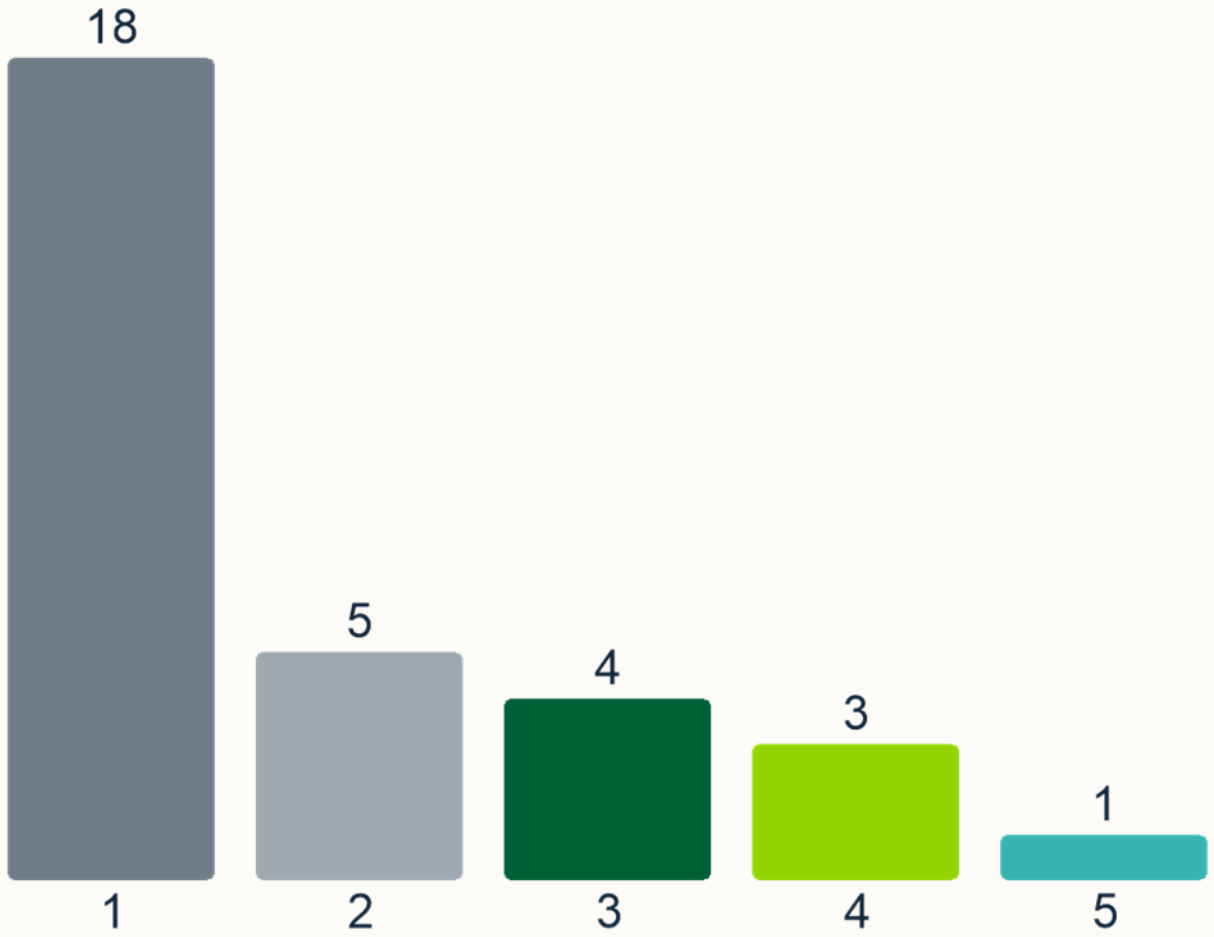
Saturation degree: 2 (blue and green)

Suppose we have a graph where one of the vertices v_x has a degree of 5 and we don't know anything else about the graph. What could the minimum saturation degree be for v_x?

Rank	Response
1	1
2	2
3	0
4	5
5	3

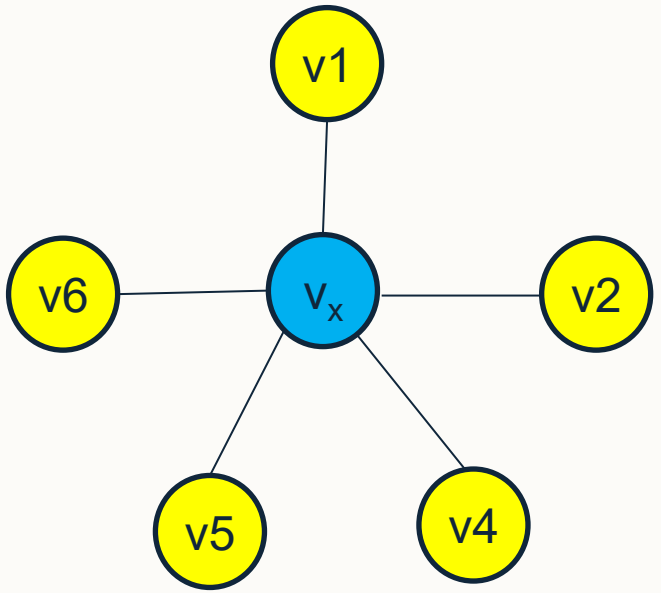
Value: 2

Correct Responses: 5



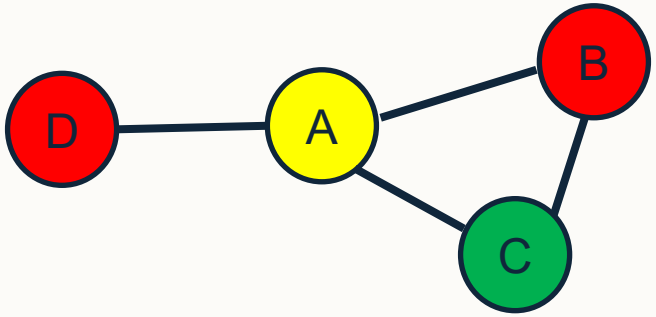
EchoPoll Answer (Q1)

- Suppose we have a graph where one of the vertices v_x has a degree of 5 and we don't know anything else about the graph. What could the minimum saturation degree be for v_x?



Graph Colouring Heuristics – Largest Degree

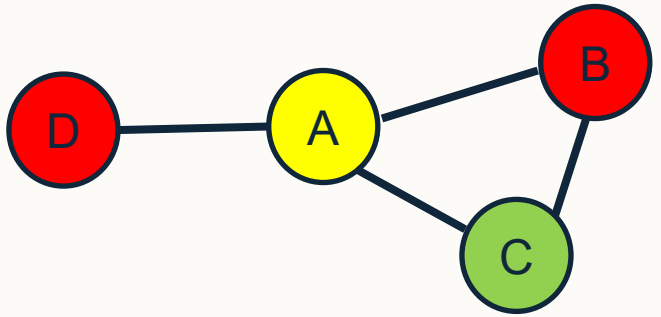
- Given a graph $G = (V, E)$, order the vertices based on their **degree**.
  - For largest degree, this ordering is highest to lowest.
- Given a list of colours, e.g. (c_1,), (c_2,), (c_3,), (c_4,), … assign the first colour to the next highest degree vertex such that no neighbour is already that colour and repeat until all vertices are coloured.
- Example – the following graph contains four edges which when ordered by degree are: $V = \{(A, 3), (B, 2), (C, 2), (D, 1)\}$.



Graph Colouring Heuristics – Saturation Degree

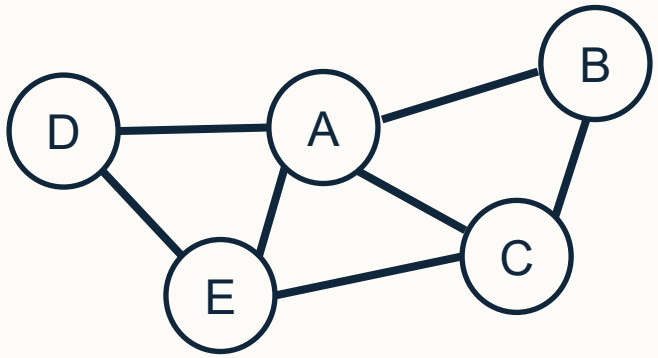
1. For a given graph $G = (V, E)$, order the vertices based on their **saturation degree** – let's assume highest to lowest for this example.
2. Given a list of colours, e.g. (c_1,), (c_2,), (c_3,), (c_4,), … assign the first colour to the next highest degree vertex such that no neighbour is already that colour and repeat until all vertices are coloured.
3. Go to (1.) until all vertices are coloured.

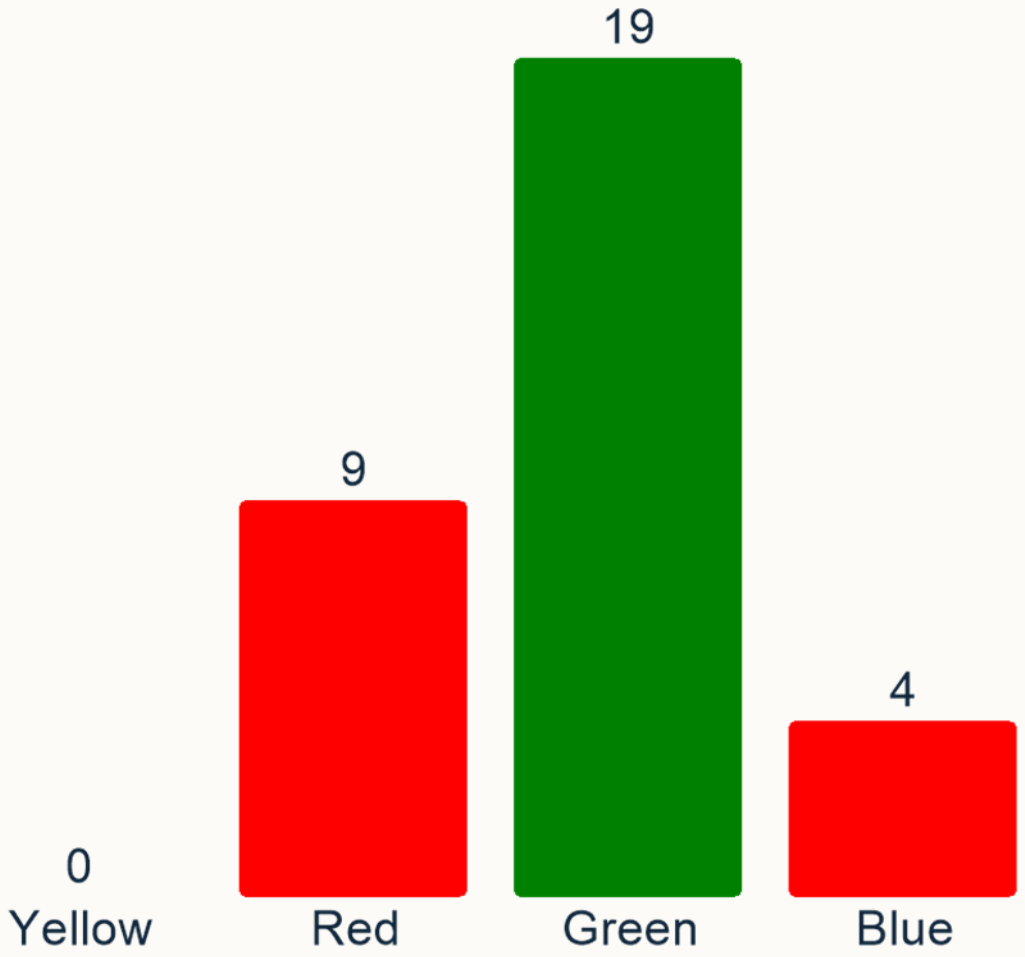
- Example – the following graph contains four edges which when ordered by saturation degree are initially: $V = \{(A, 0), (B, 0), (C, 0), (D, 0)\}$.
- See the lecture recording for a step-by-step guide on how to colour this graph.



Imagine a similar graph (on the slide) which we colour using the SD heuristic. Given the same colour ordering (Y/R/G/B) which colour will vertex D be given?

A. Yellow
B. Red
✓C. Green
D. Blue

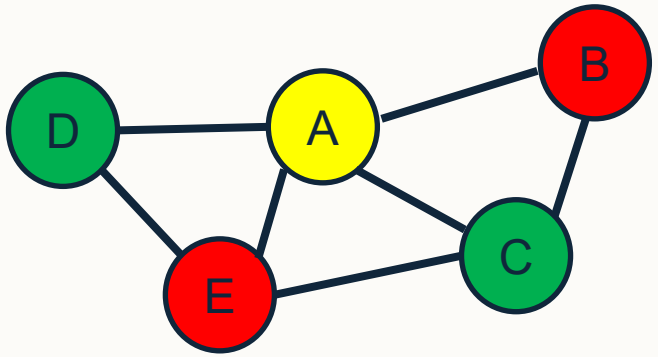




EchoPoll Answer (Q2)

1. For a given graph $G = (V, E)$, order the vertices based on their **saturation degree** – let's assume highest to lowest for this example.
2. Given a list of colours, e.g. (c_1, 🟡), (c_2, 🔴), (c_3, 🟢), (c_4, 🔵), … assign the first colour to the next highest degree vertex such that no neighbour is already that colour and repeat until all vertices are coloured.
3. Go to (1.) until all vertices are coloured.

See the lecture recording for a step-by-step guide on how to colour the graph.



Examination Timetabling Problem

- Given a set of exams E (e1, e2, …, eE) taken by students S (s1, s2, …, sS), schedule each student and exam into a limited time period T (t1, t2, …, tT) within a limited number of rooms R (r1, r2, …, rR).

- **Hard constraints** include exams taken by the same student cannot be assigned to the same time period, and the capacity of a room cannot be exceeded, amongst hundreds of others…

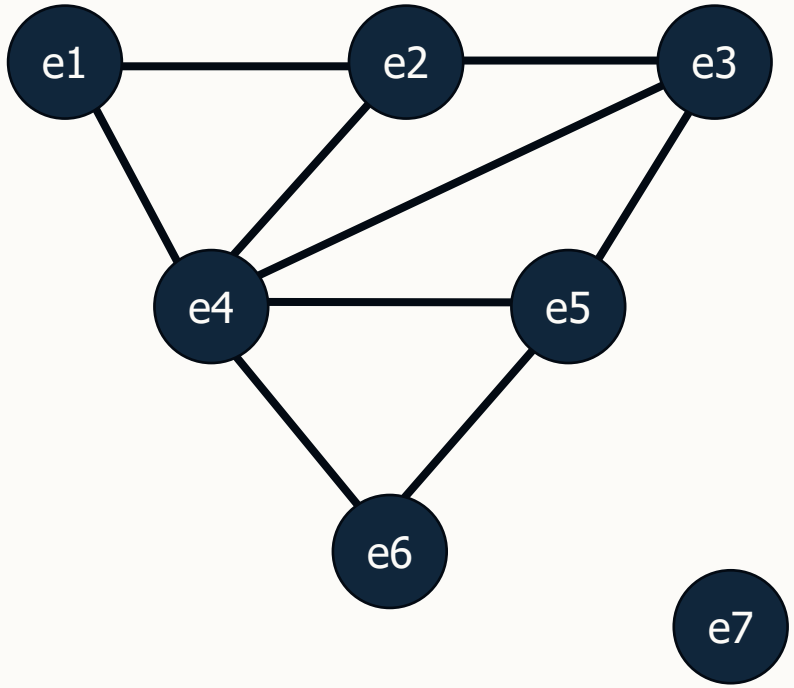
- **Soft constraints** include time separation between exams, and scheduling exams with large cohorts earlier.

Designing a Local Search Metaheuristic for Examination Timetabling

- **Representation** can use list of pairs of integers, each representing the time period assignment and room assignment. The size of the list is the number of examinations, where each time period is an integer in the range [1,T] and each room has an integer value in the range [1,R].
- **Initialisation** could randomly assign each exam to a time period and room.
- **Objective Function** a measure of hard and soft constraint violations.
- **Neighbourhood operator(s)** are perturbation operators:
  - OP1: randomly choose an exam and assign it to a different time period.
  - OP2: randomly choose an exam and assign it to a different room.
  - OP3: randomly choose an exam and assign it to both a different time period and different room.
  - All of the above could be parameterised to pick x random exams.
- Also possible to design other search methods (e.g. Iterated Local Search/Simulated Annealing) using appropriate components.

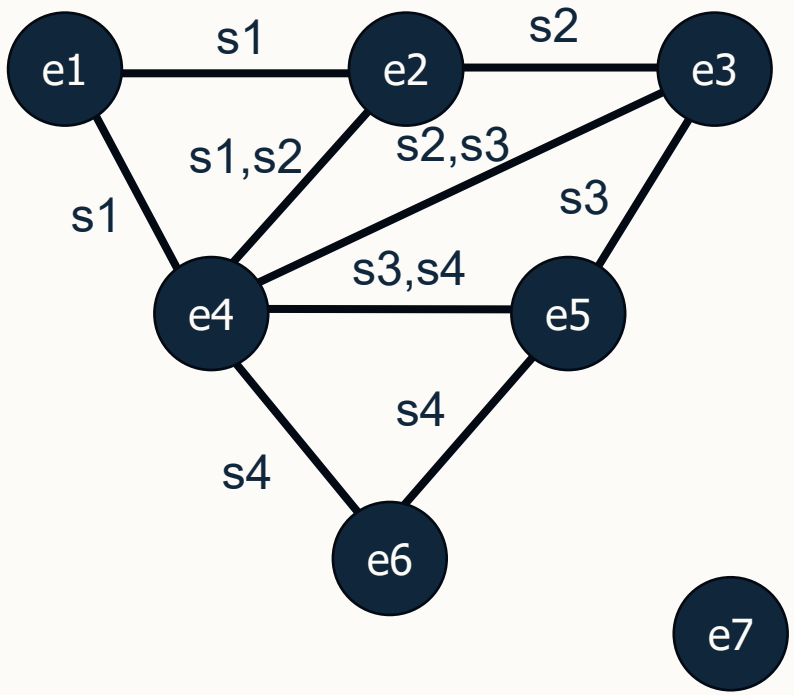
Modelling Examination Timetabling as a Graph 1

- Consider a simplified exam timetabling problem allocating exams to time periods with 7 exams $E = (e1, e2, \ldots, e7)$ and 5 students $S = (s1, s2, \ldots, s5)$ taking the exams:
  - $s1: e1, e2, e4$
  - $s2: e2, e3, e4$
  - $s3: e3, e4, e5$
  - $s4: e4, e5, e6$
  - $s5: e7$

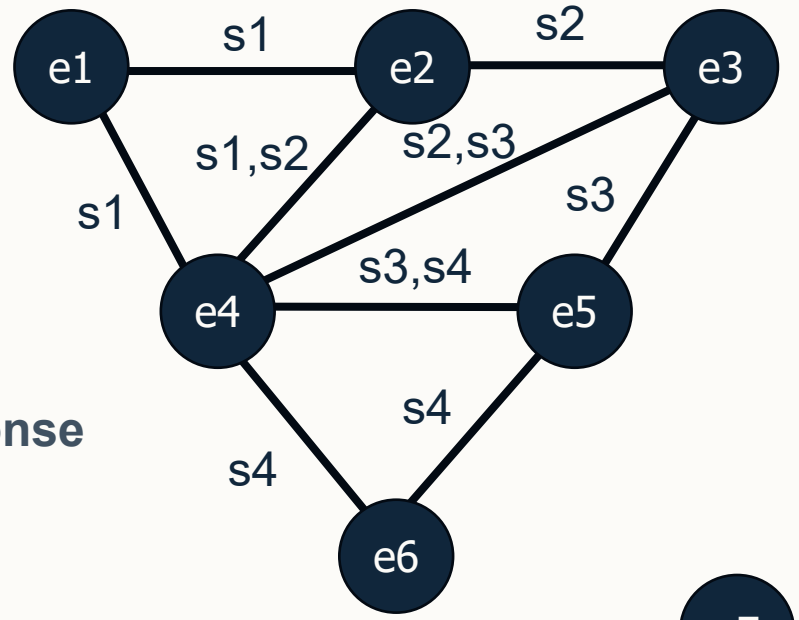


Modelling Examination Timetabling as a Graph 2

- **Nodes** represent exams: $E = (e1, \ldots, e7)$.
- **Edges** connect nodes (exams) to represent exams with common students.
- **Colours** represent different time periods.
- **Objective** is to colour the graph using the minimum number of colours.
- **Question: how many colours are needed to colour this graph?**



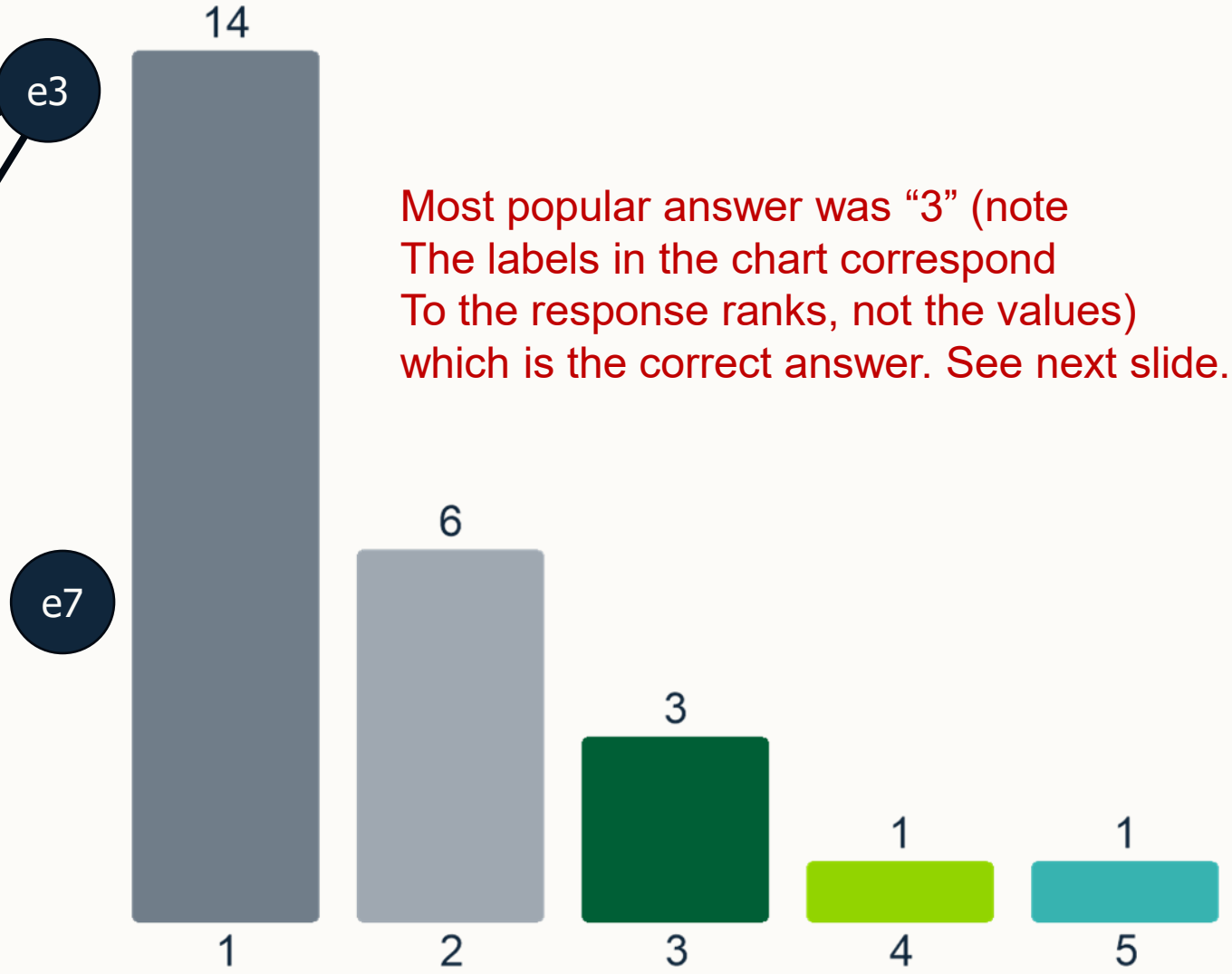
What are the minimum number of colours needed to colour this graph?



Rank	Response
1	3
2	4
3	5
4	6
5	2

Value: 3

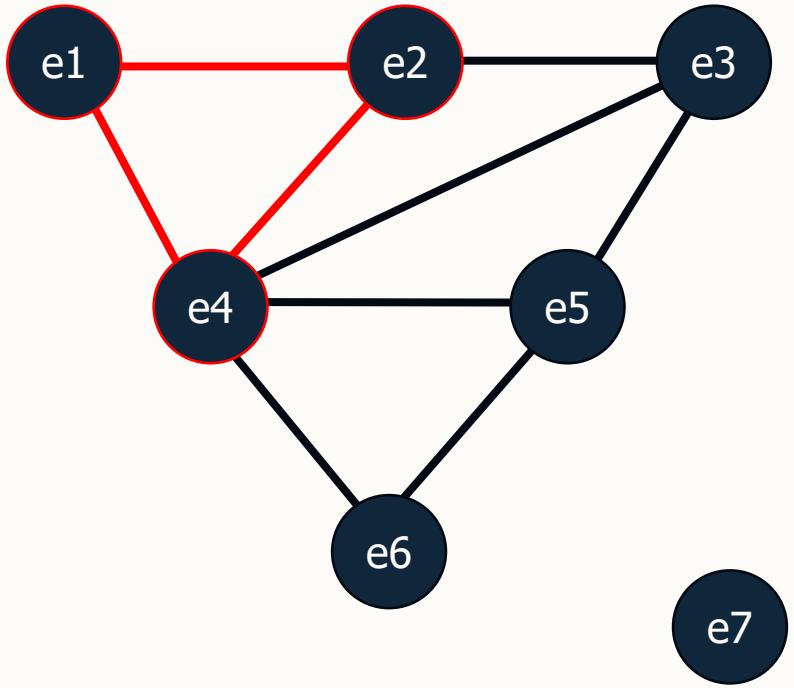
Correct Responses: 14



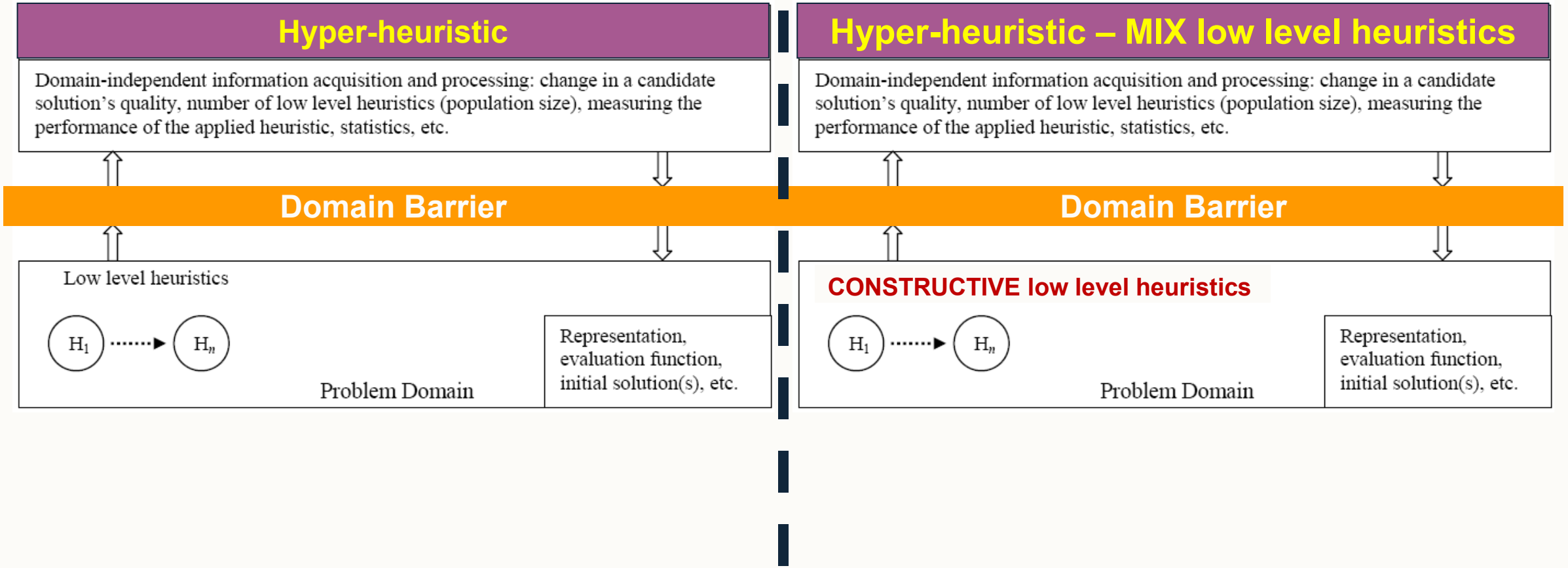
Most popular answer was “3” (note
The labels in the chart correspond
To the response ranks, not the values)
which is the correct answer. See next slide.

EchoPoll Answer (Q3)

- Minimum colours can be found by identifying the largest clique – hence answer is 3.

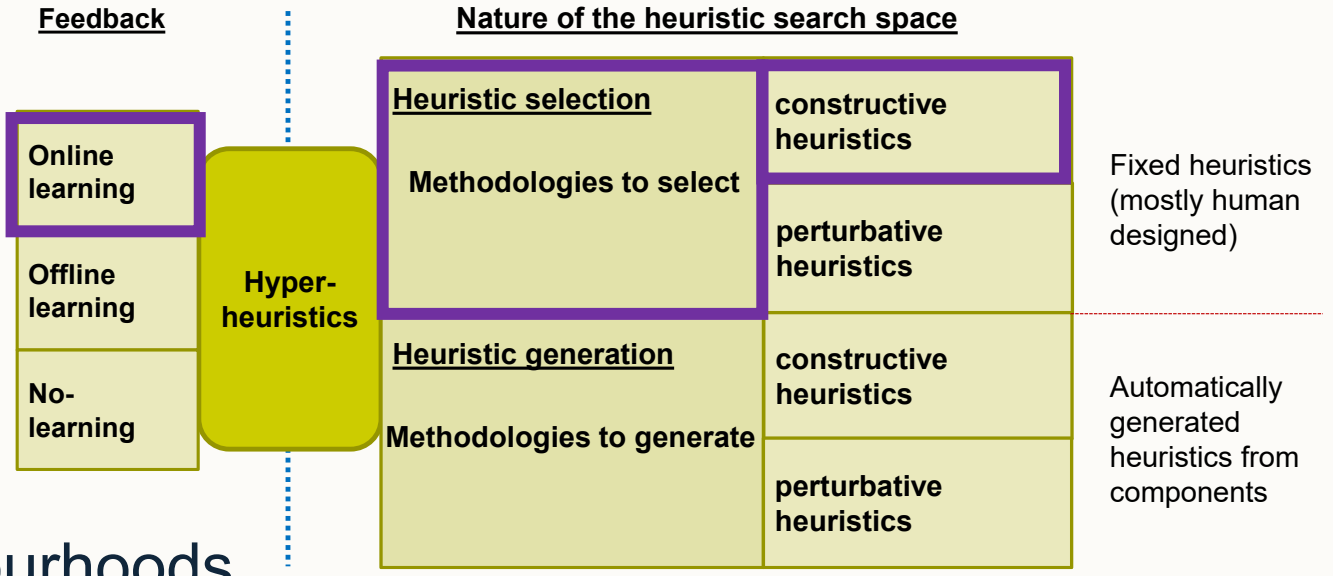


A Hyper-heuristic Framework for Selecting Constructive Heuristics



Graph-based Hyper-heuristics (GHH)

- An online-learning based hyper-heuristic selecting from low-level constructive heuristics.
- A general framework for constructing solutions.
- Defined 6 constructive graph heuristics.
- Uses a list of these heuristics $h1 = \{h_1, h_2, \ldots, h_k\}$ and employs a swap operator to define neighbourhoods.
- Applies each of the heuristics in list order to construct a solution.



E. Burke, B. McCollum, A. Meisels, S. Petrovic, and R. Qu. A graph-based hyper-heuristic for educational timetabling problems. European Journal of Operational Research, 176(1):177-192 [Link to the paper]

Graph Colouring Heuristics for the GHH

- **Largest degree (LD)** aims to schedule exams with the most potential conflicts first.
- **Saturation degree (SD)** prioritises scheduling exams with the highest number of available time slots (colours).
- **Colour degree (CD)** opposite of saturation degree.
- **Largest enrolment (LE)** schedules exams with the highest number of students first.
- **Largest Weighted Degree (LWD)** combines LD and LE by assigning weights to each edge as the number of students involved.
- **Random Ordering (RO)** orders exams that are not yet scheduled randomly.

Graph Colouring Heuristics for the GHH

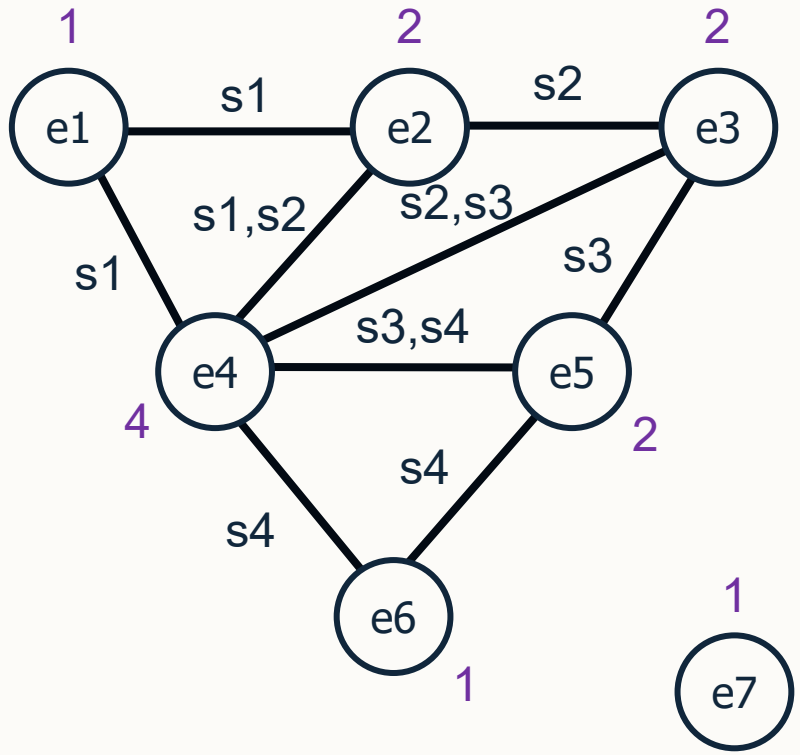
- **Largest degree (LD)**
  - aims to schedule exams with the most potential conflicts first.
- **Saturation degree (SD)**
  - prioritises scheduling exams with the fewest number of available time slots.
- **Colour degree (CD)**
  - opposite of saturation degree.
- **Largest enrolment (LE)**
  - schedules exams with the highest number of students first.
- **Largest Weighted Degree (LWD)**
  - combines LD and LE by assigning weights to each edge as the number of students involved.
- **Random Ordering (RO)**
  - orders exams that are not yet scheduled randomly.

GHH Example (Pass 1)

- $h1 = \{LE, SD, LD, LW, \ldots\}$
- Exams $= \{e1, e2, e3, e4, e5, e6, e7\}$
- N=2

1. Choose the next heuristic from *h1* as *LE*.
2. Order the remaining exams by LE.
   exams = { e4, e2, e3, e5, e1, e6, e7 }
3. For I = 0 to N-1 schedule exams[i]

Exam slots = [<e4>, <e2>]

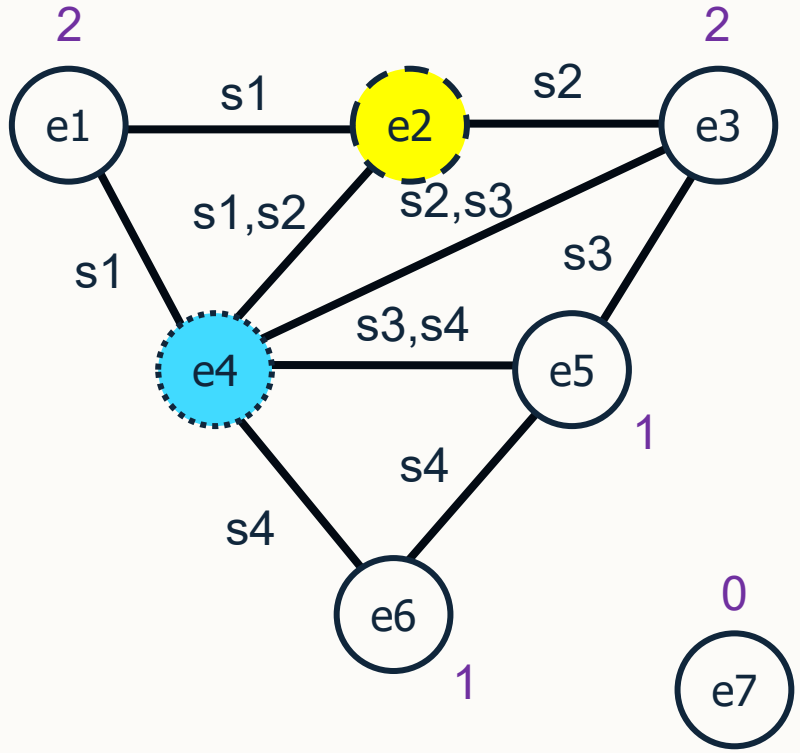


GHH Example (Pass 2)

- $h1 = \{SD, LD, LW, \ldots\}$
- Exams ={e1, e3, e5, e6, e7}
- N=2

1. Choose the next heuristic from *h1* as *SD*.
2. Order the remaining exams by SD.
   exams = { e1, e3, e5, e6, e7 }
3. For I = 0 to N-1 schedule exams[i]

Exam slots = [<e4>, <e2>, <e1,e3>]

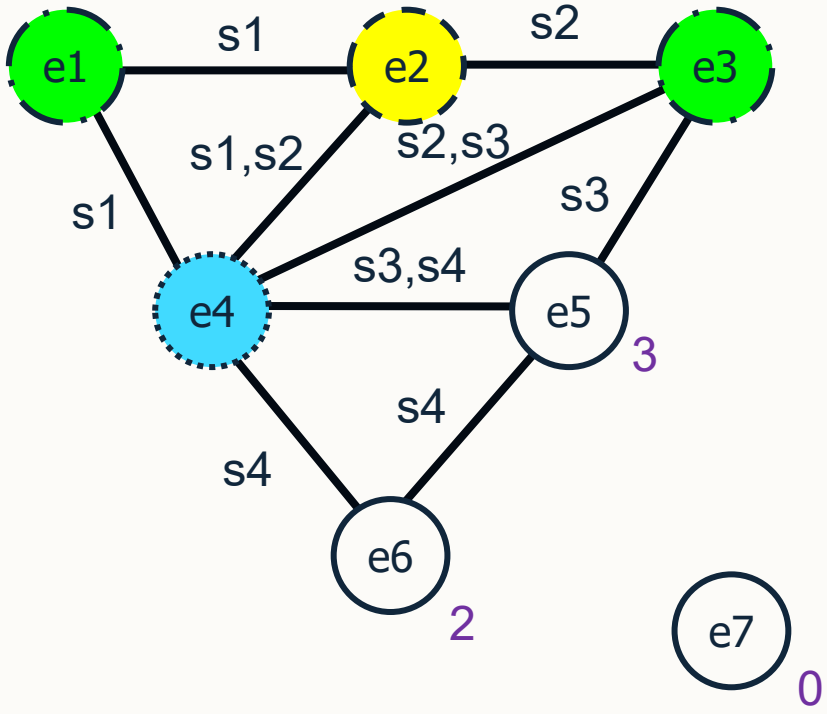


GHH Example (Pass 3)

- $h1 = \{LD, LW, \ldots\}$
- Exams = {e5, e6, e7}
- N=2

1. Choose the next heuristic from *h1* as *LD*.
2. Order the remaining exams by LD.
   exams = { e5, e6, e7 }
3. For I = 0 to N-1 schedule exams[i]

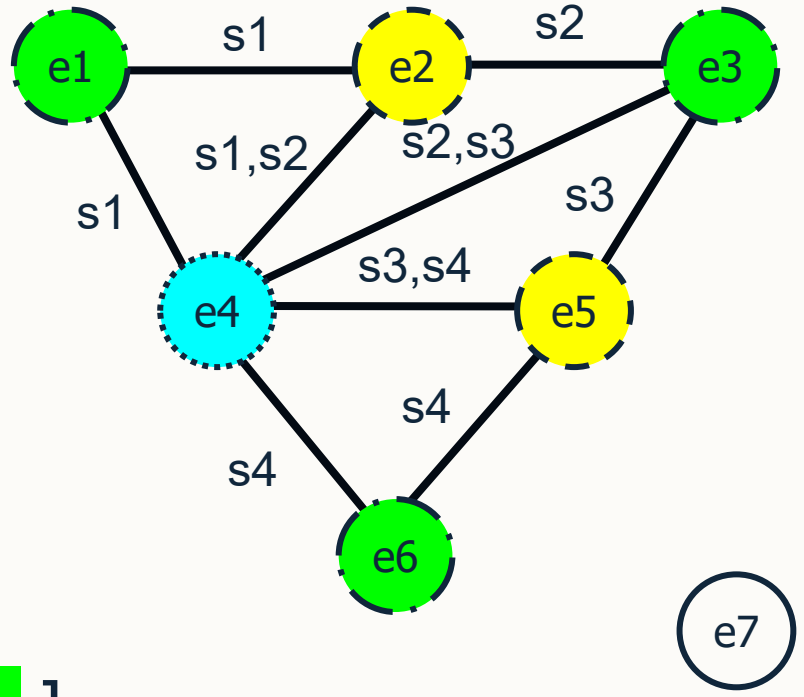
Exam slots = [<e4>, <e2, e5>, <e1, e3, e6>]



GHH Example (Pass 4)

- $h1 = \{LW, \ldots\}$
- Exams = { e7 }
- N=2

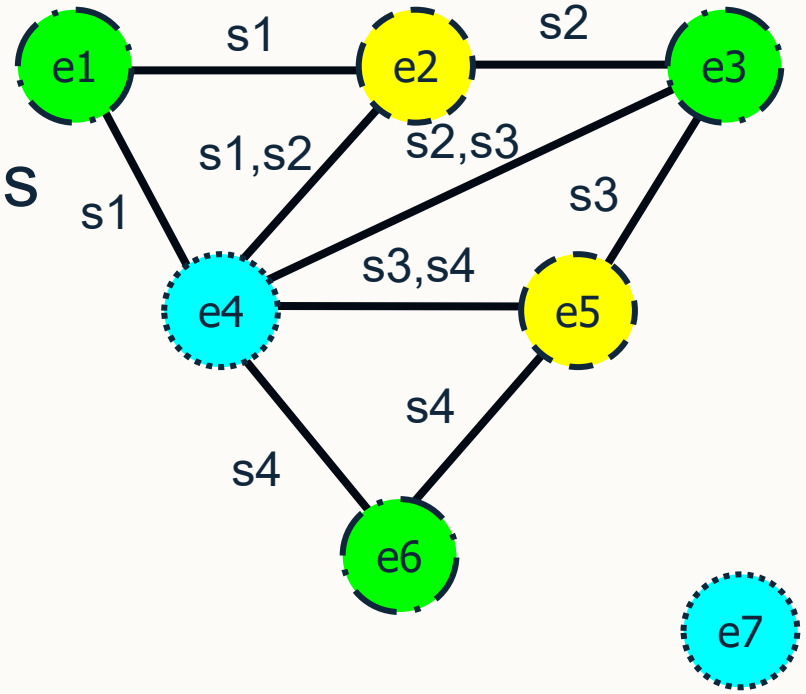
1. Choose the next heuristic from *h1* as *LW*.
2. Order the remaining exams by LW.
   exams = { e7 }
3. For I = 0 to N-1 schedule exams[i]



Exam slots = [<e4, e7>, <e2, e5>, <e1, e3, e6>]

GHH Example – Complete Solution

- Exam slots = [<e4,e7>, <e2, e5>, <e1, e3, e6>]
- We have now constructed a complete and feasible solution with 3 colours (optimal).



- For more difficult problems we may repeat this process multiple times (5*|E| in paper).
- Example:
  - h1_1 = { LE, SD, LD, LW }
  - h1_2 = { LE, LD, SD, LW }
  - h1_3 = { LD, LE, DS, LW }
  - …

GHH Results

- GHH was evaluated using Carter's benchmark with different pools of low-level heuristics and found a combination of SD, CD, LW to be the best.

Problem	SL	SLR	SCL	SCLR	SCLx	SCLxR	SCLxR(10*)
car91	5.78	5.67	5.52	**5.36**	5.65	5.43	5.39
car92	4.76	4.68	4.21	**4.14**	4.53	4.78	4.63
ear83	38.8	38.57	38.68	38.5	**37.92**	38.22	38.03
hec92	12.35	12.27	12.30	12.81	12.39	12.25	***12.11***
kfu93	16.21	15.79	15.37	15.23	15.67	15.2	***15.12***
lse91	12.17	11.36	12.09	11.93	11.56	**11.33**	***11.33***
sta83	164.01	163.5	164.06	163.31	**158.19**	160.19	159.32
tre92	9.15	9.13	8.87	9.08	**8.75**	9.03	8.97
ute92	29.49	29.17	28.27	28.19	**28.01**	28.21	28.11
uta93	4.12	4.03	4.05	3.98	**3.88**	3.95	***3.78***
york83	44.54	42.67	42.44	42.37	**41.37**	42.01	41.52

Costs of solutions obtained by GHH upon a different number of heuristics (S: saturation degree, L: largest degree, C: color degree; R: random ordering, Lx: largest weighted)

Genetic Programming

- Genetic programming
- Genetic operators for GP

Genetic Programming (GP)

- Goal of **GP** is to get a computer to do a given task without telling it how to do it.
- Similar in many ways to machine learning methods but uses an evolutionary process to create new programs.
- Uses parse trees as a flexible representation of a computer program.

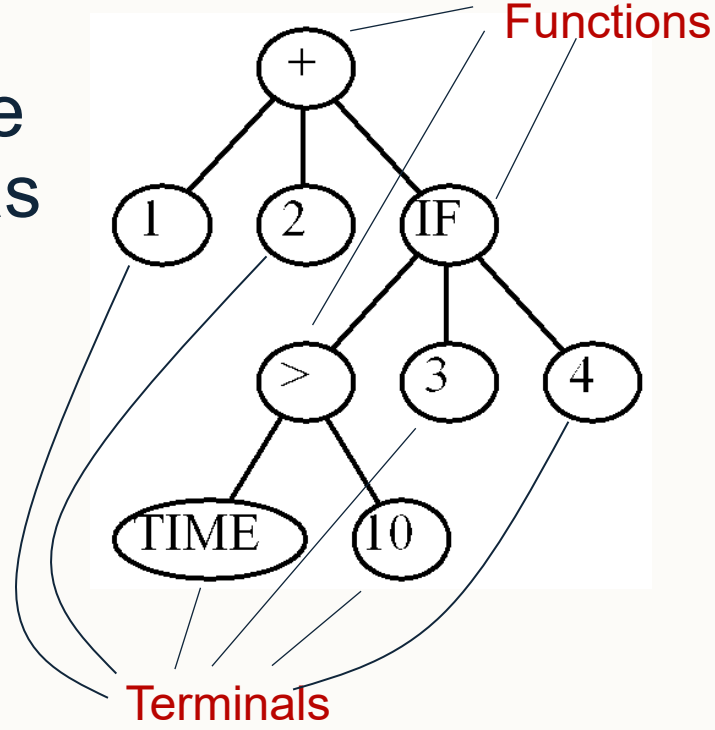
- *"It is difficult, unnatural, and overly restrictive to attempt to represent hierarchies of dynamically varying size and shape with fixed length character strings." "For many problems in machine learning and artificial intelligence, the most natural representation for a solution is a computer program."* [Koza, 1994]

Example of a known program

```
int foo (int time) {
    int temp1, temp2;
    if (time > 10)
        temp1 = 3;
    else
        temp1 = 4;
    temp2 = temp1 + 1 + 2;
    return (temp2);
}
```

Using Trees to Represent Programs

- If we run the program across different “time” inputs, we can get a mapping of inputs to outputs.
- Using GP, can we then create a program that is the same as the known program?
- Can construct the following tree to compute the same outputs as the known program.




```
(+ 1 2 (IF (> TIME 10) 3 4))
```

Time	Output
0	7
1	7
2	7
3	7
4	7
5	7
6	7
7	7
8	7
9	7
10	7
11	6
12	6
13	6
14	6

Genetic Operators for GP

- GP is an evolutionary algorithm and contains the same components as we saw before:
  - **Initialisation** of the population could be by randomly generating a program for each individual/program.
  - **Crossover** can be used to combine two solutions to produce new solutions/programs.
  - **Mutation** can be used to create new solutions/programs making some small change.

Random Generation of Programs

- **Example**: randomly generate a program that takes two arguments and uses the basic arithmetic operators:
  - **Function set (internal nodes)**: { +, -, *, / }
  - **Terminal set (leaf nodes):** { X, Y, integer_values }

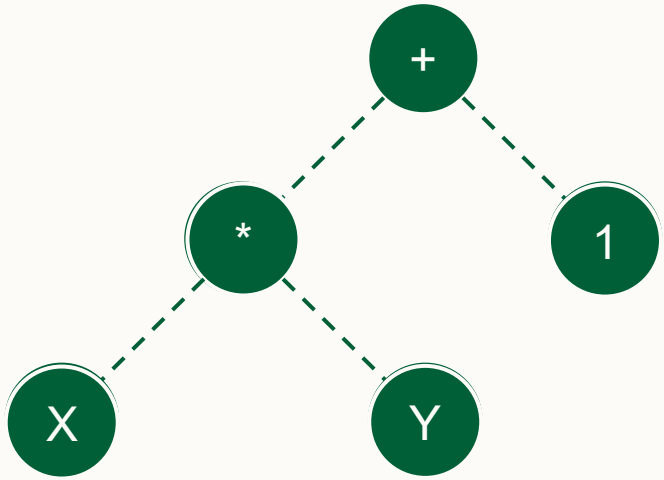
- Random generation randomly selects a function or terminal to represent the program.

- If a function is chosen, we recursively generate a random program until all leaf nodes are terminals.

Random Generation Example

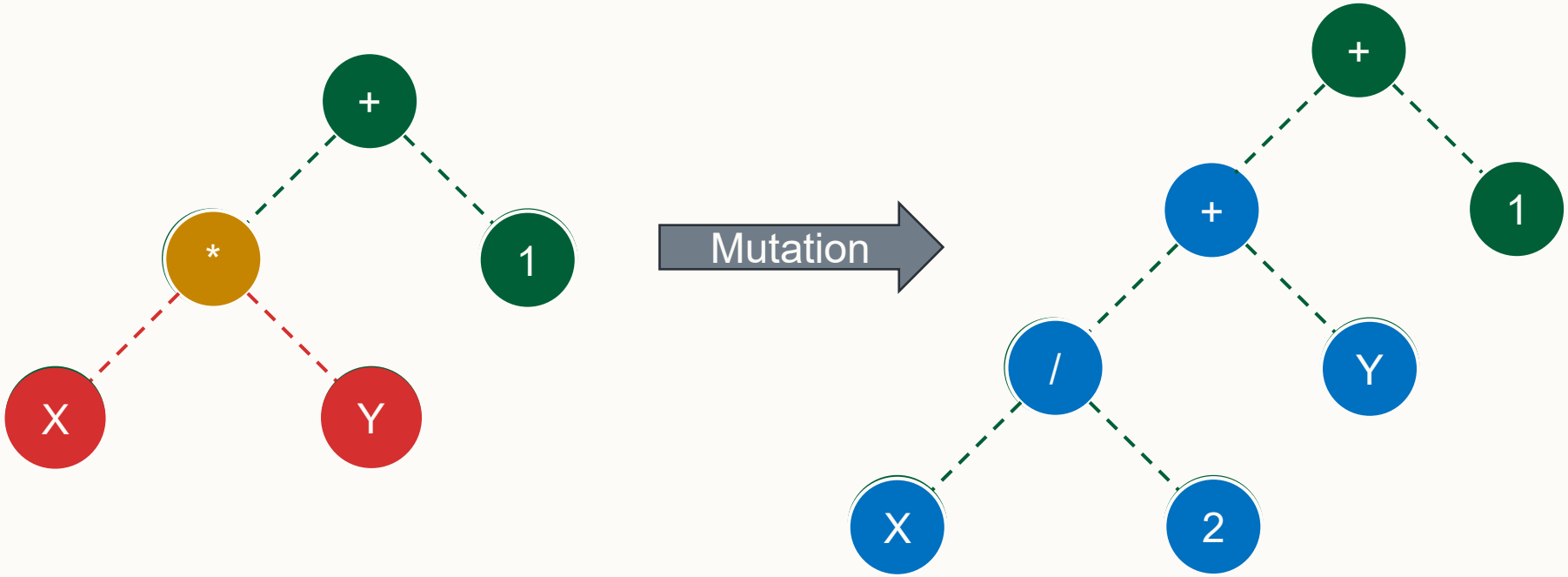
- **Function set (internal nodes)**: { +, -, *, / }
- **Terminal set (leaf nodes):** { X, Y, integer_values }

1. Chosen binary `+`
2. Recurse on each child node
   a. Chosen binary `*`
   b. Recurse on each child node
      i. Chosen terminal `X`
      ii. Chosen terminal `Y`
   c. Chosen terminal `1`



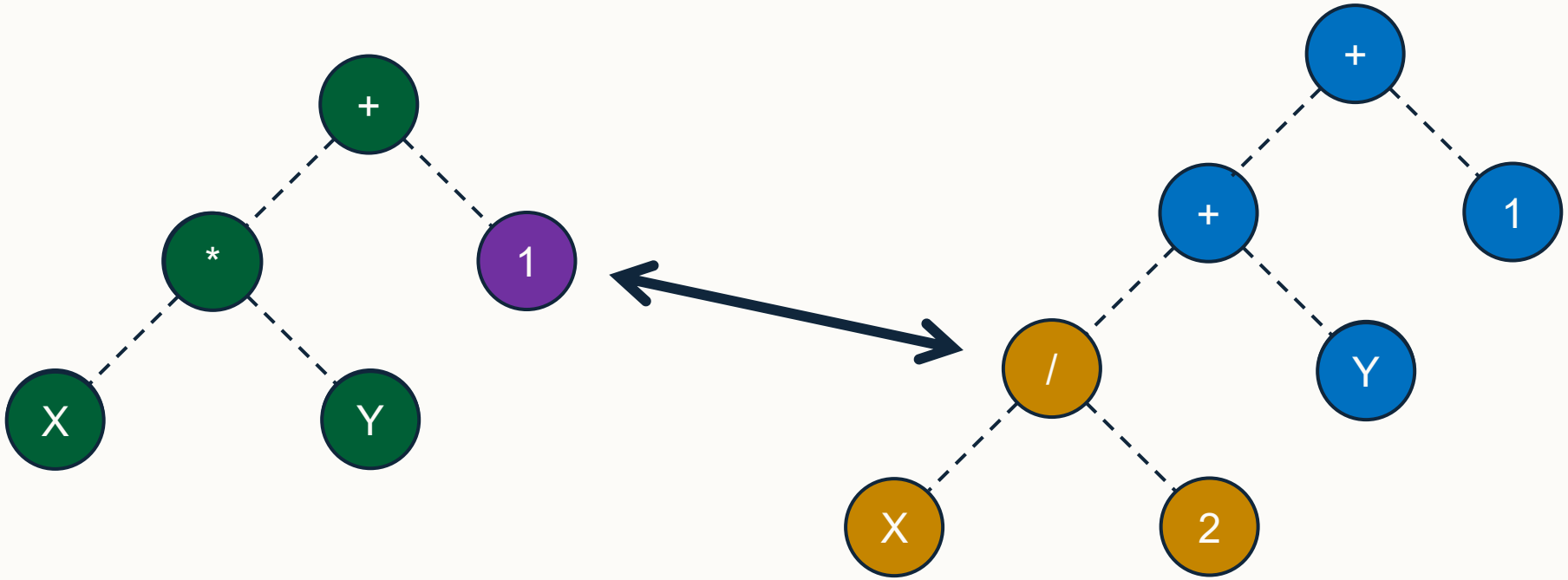
Mutation in GP

1. Select a random node and remove it from the tree.
   - If the node was an internal node, then remove all children as well.
2. Insert a **randomly generated** program in place of the node just deleted.



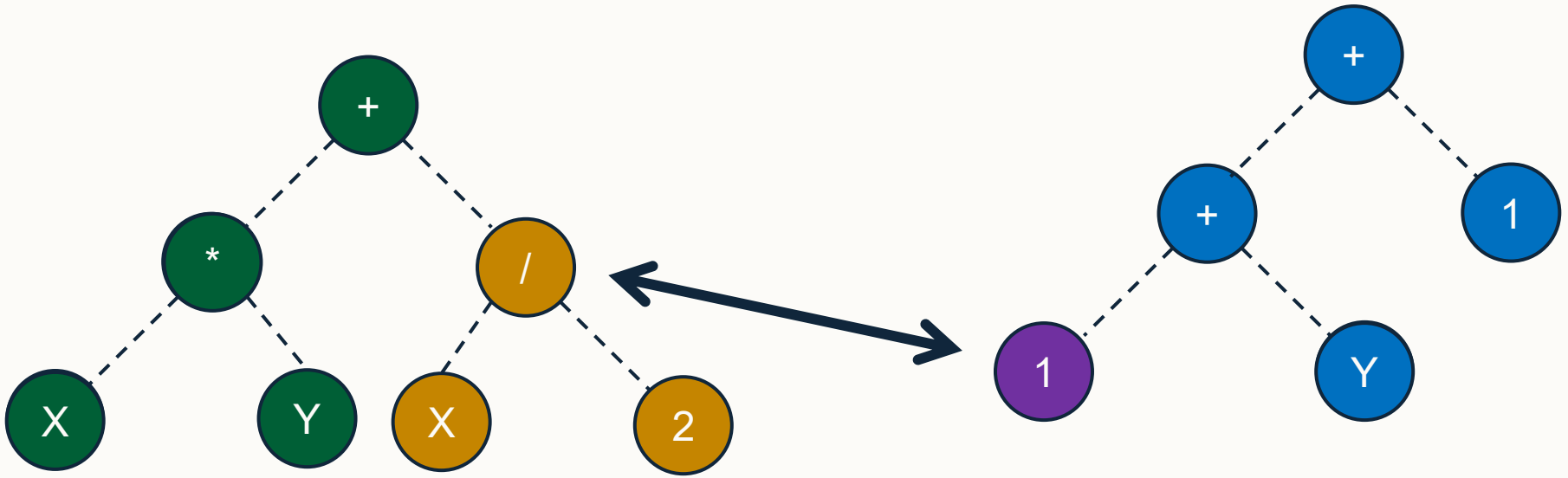
Crossover in GP (1)

- For two programs p1 and p2, select a subtree from each program and swap them.



Crossover in GP (2)

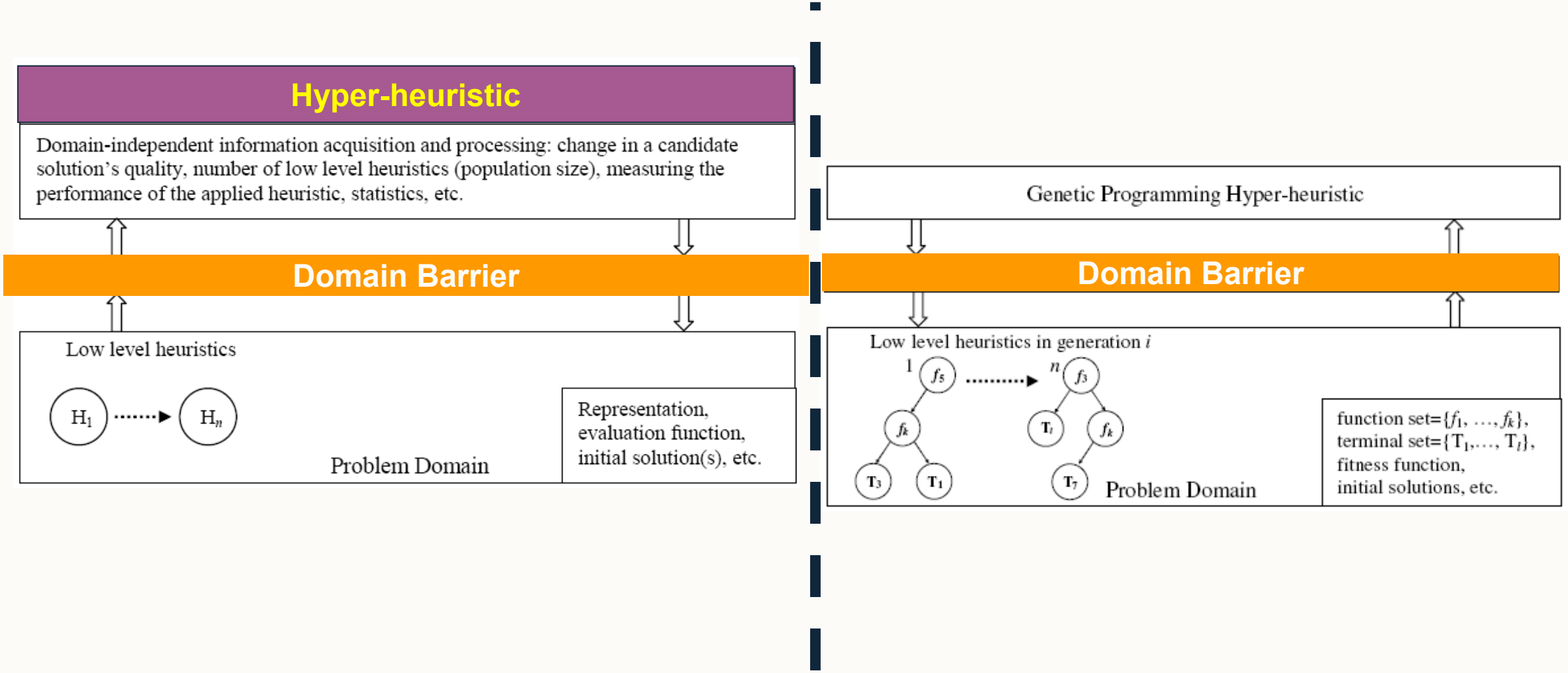
- For two programs p1 and p2, select a subtree from each program and swap them.



Libraries/APIs for GP (Additional Reading)

- ECJ: http://cs.gmu.edu/~eclab/projects/ecj/
- TinyGP: http://cswww.essex.ac.uk/staff/rpoli/TinyGP/
- GEVA (grammatical evolution): http://ncra.ucd.ie/Site/GEVA.html
- Cartesian GP resources: http://www.cartesiangp.co.uk/resources.html

A Hyper-heuristic Framework for GP



Reusable and Disposable Heuristics

- Two different approaches to generative hyper-heuristic design.

- **Disposable heuristics:** At each run of the HH we try to generate good heuristics from function and terminal sets for solving the given problem/instance.

- **Reusable heuristics:** Train the system to generate good heuristics for solving a particular problem, then re-use the system for solving other instances of the same problem making use of the pre-generated set of low-level heuristics.

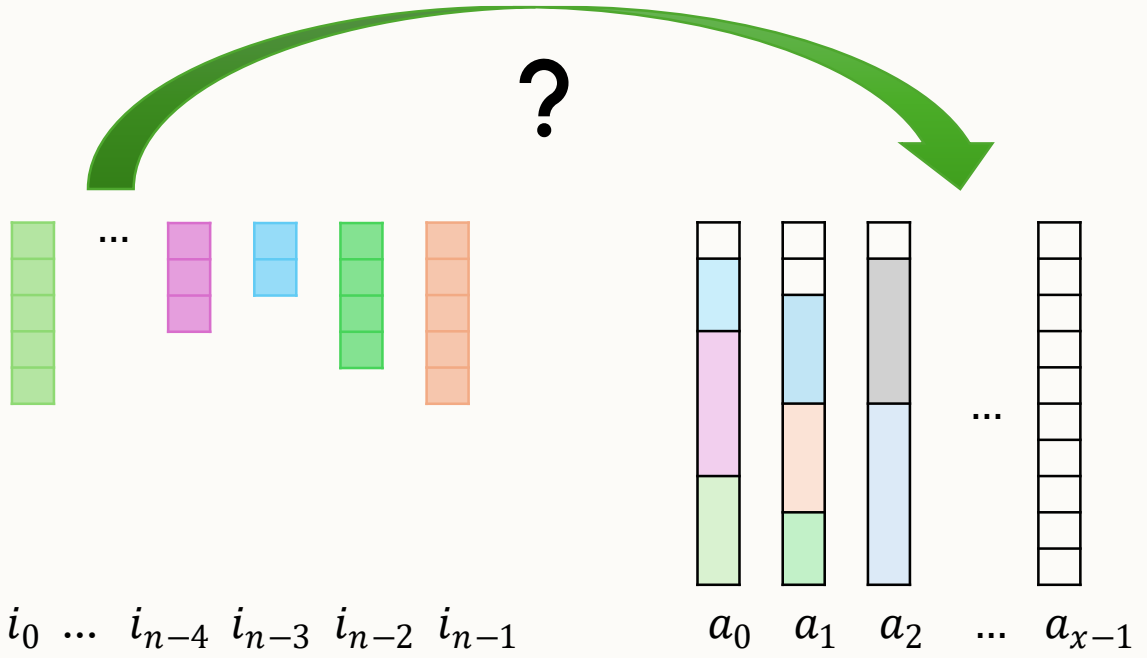
Generation Hyper-heuristics

Solving Bin Packing Problems with Genetic Programming

From PhD Thesis of Matthew Hyde (2010) [Link to the thesis]

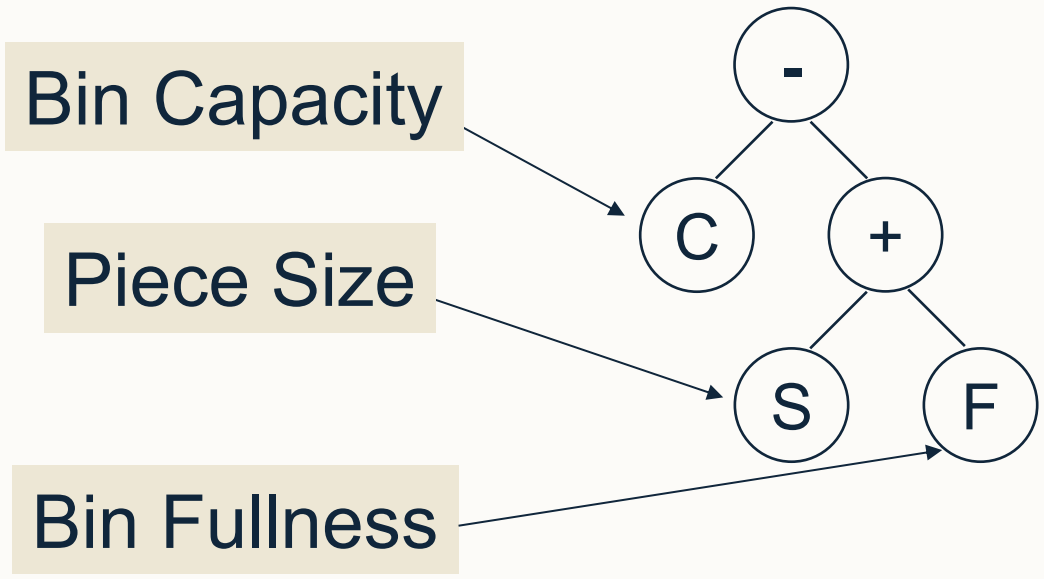
Recap – 1D Offline Bin Packing (lec2)

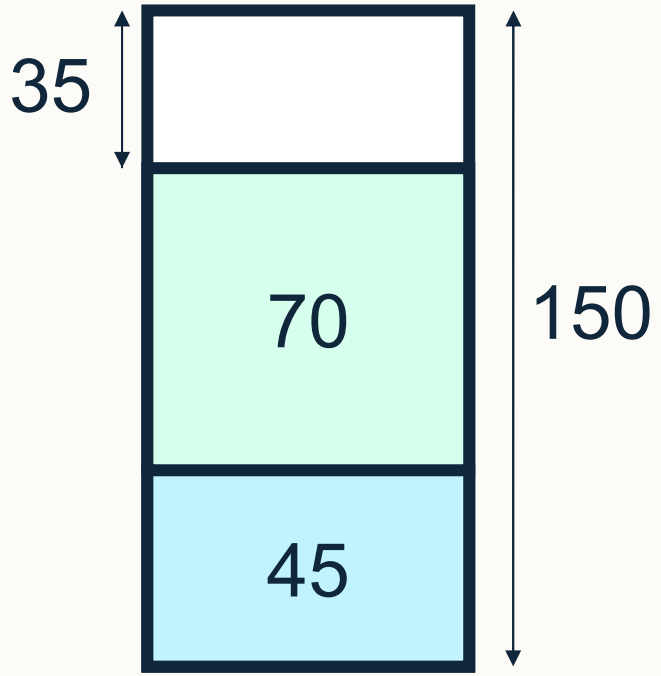
- Given a set of items I, $|I| = n$, containing items of various sizes s_x, and an unlimited number of bins a_x, each with fixed capacity C, where $0 \leq x \leq n$…
- …what is the minimum number of bins required to pack all items without exceeding the capacity of any of the bins?



GP for Bin Packing (1)

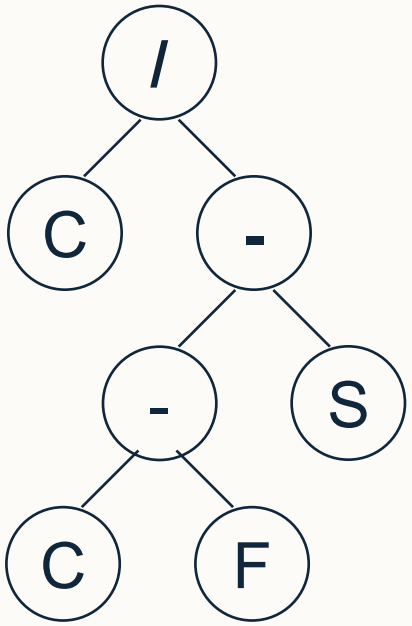
- Basic idea is to generate a program to calculate some score/metric of the open bins to decide which bin to place the next item in.
- The following program/tree is a way of calculating the space left in the bin after packing the new item.

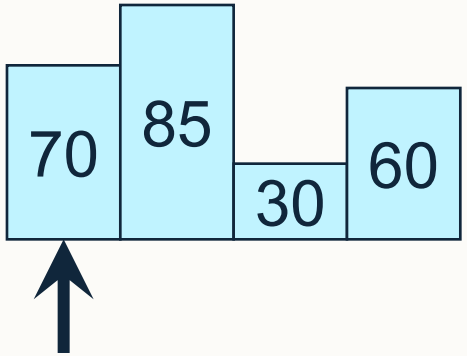


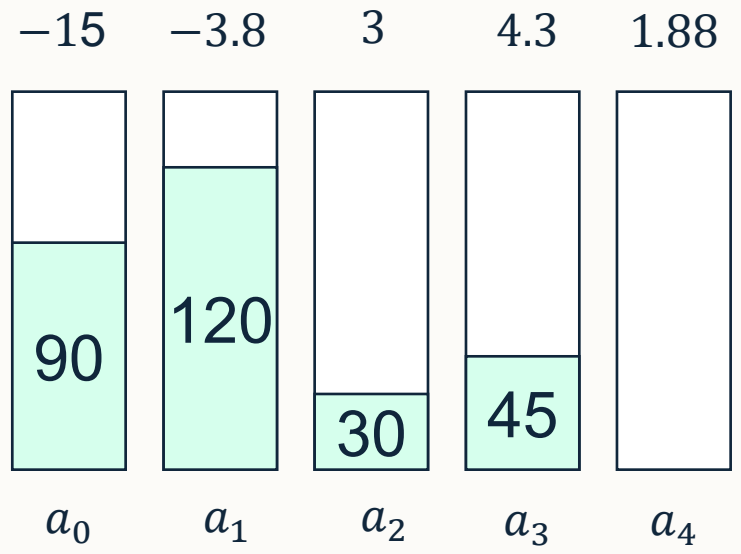


GP for Bin Packing (2)

- Imagine the following tree where *C* is a fixed capacity (*C*=150), *S* is the size of the item to be packed, and *F* is the bin fullness.
- Items i_0 through i_3 are already packed with 4 items remaining with size 70, 85, 30, and 60 respectively.

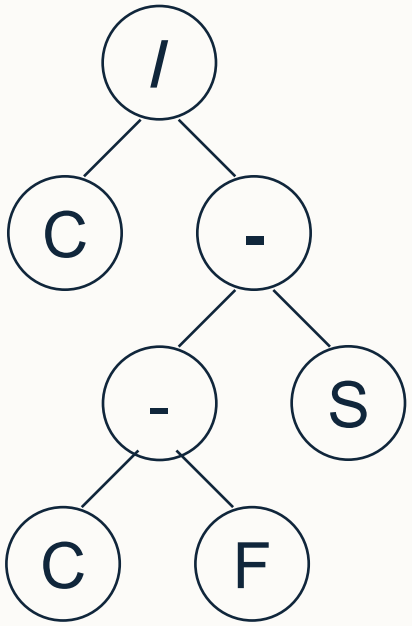


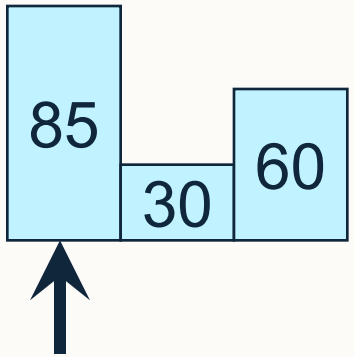


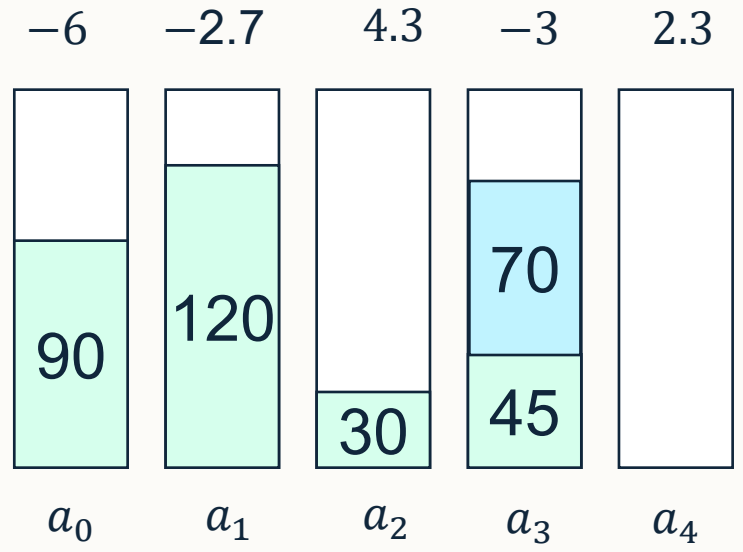


GP for Bin Packing (3)

- Imagine the following tree where *C* is a fixed capacity (*C*=150), *S* is the size of the item to be packed, and *F* is the bin fullness.
- Items i_0 through i_3 are already packed with 4 items remaining with size 70, 85, 30, and 60 respectively.

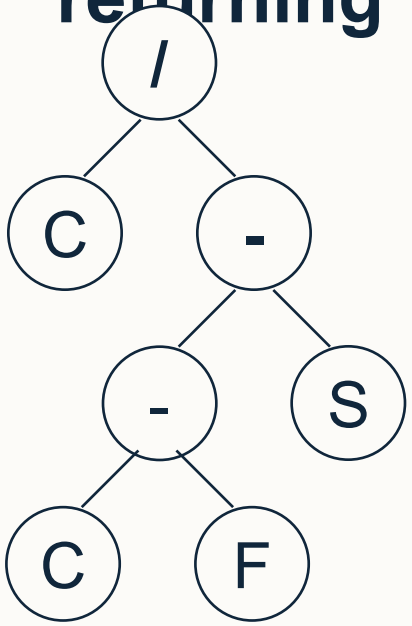


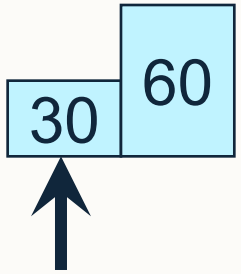


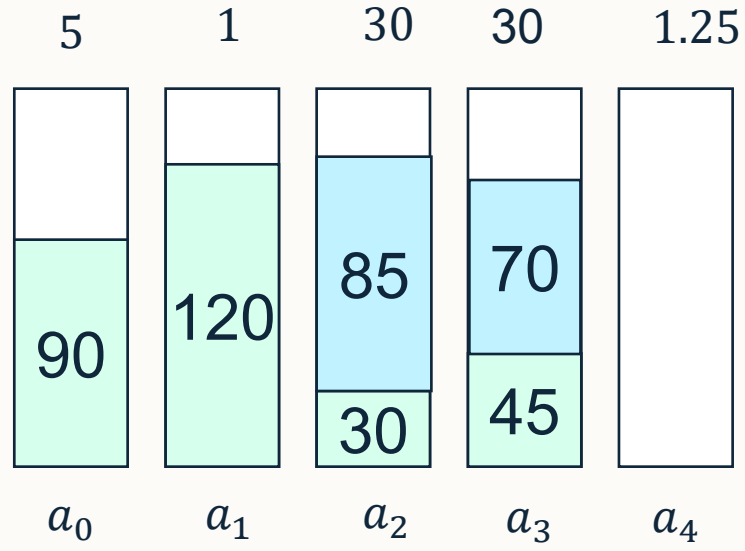


GP for Bin Packing (4)

- Imagine the following tree where *C* is a fixed capacity (*C*=150), *S* is the size of the item to be packed, and *F* is the bin fullness.
- Items i_0 through i_3 are already packed with 4 items remaining with size 70, 85, 30, and 60 respectively. **Note the `/` is a safe divide returning 1 if the denominator is 0.**

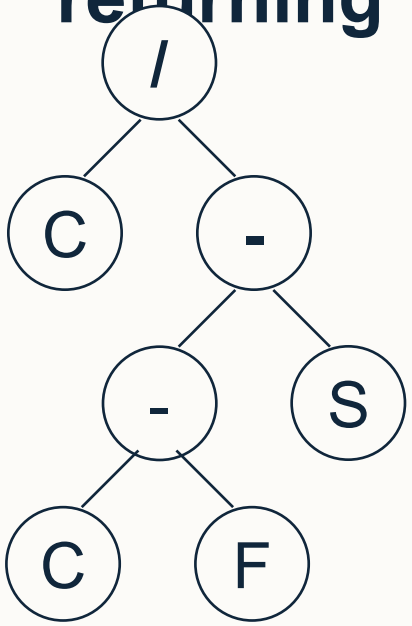


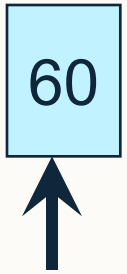


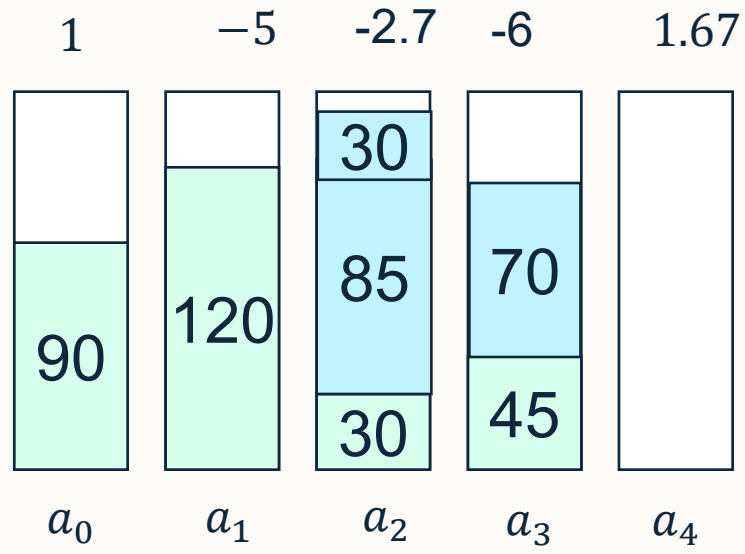


GP for Bin Packing (5)

- Imagine the following tree where *C* is a fixed capacity (*C*=150), *S* is the size of the item to be packed, and *F* is the bin fullness.
- Items i_0 through i_3 are already packed with 4 items remaining with size 70, 85, 30, and 60 respectively. **Note the `/` is a safe divide returning 1 if the denominator is 0.**

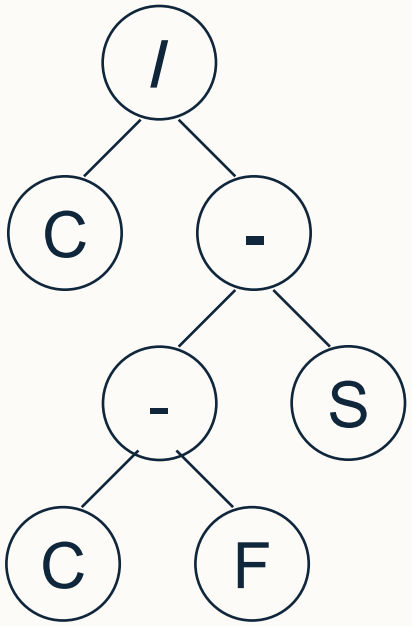


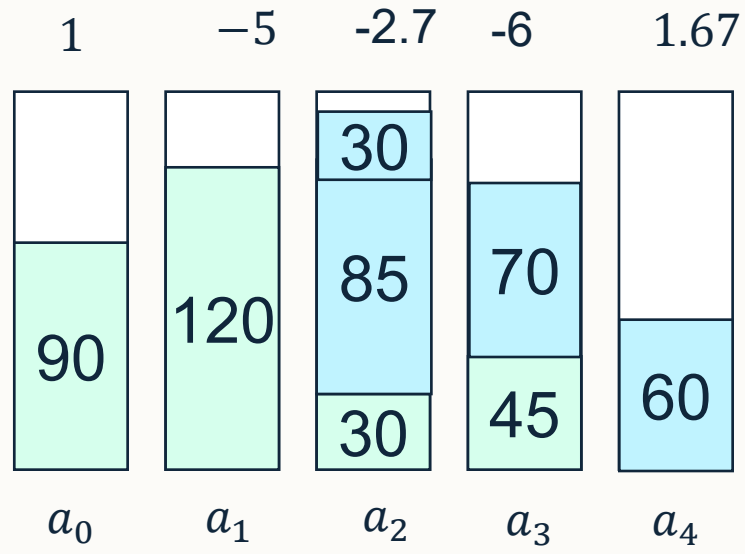




GP for Bin Packing (6)

- Imagine the following tree where *C* is a fixed capacity (*C*=150), *S* is the size of the item to be packed, and *F* is the bin fullness.
- Items i_0 through i_3 are already packed with 4 items remaining with size 70, 85, 30, and 60 respectively.





Evaluation against best-in-literature

- Generality of the approach was tested over 1D Bin Packing, 2D Knapsack and Bin Packing, and 3D Knapsack and Bin Packing problems.

Bin Packing:

Dimensions	Instance Name	Percent Improvement
1D	Uniform	0
	Hard	-0.4
2D	Beng	0
	Ngcut	0
	Gcut	-1.2
	Cgcut	0
3D	Thpack9	-2.3

Knapsack:

Dimensions	Instance Name	Percent Improvement
2D	Okp	**+1.2**
	Wang	0
	Ep30	-0.9
	Ep50	**+0.4**
	Ep100	-2.6
	Ep200	**+2.4**
	Ngcut	-4.3
	Gcut	**+1.2**
	Cgcut	-1.7
3D	Ep3d20	**+13.0**
	Ep3d40	**+10.2**
	Ep3d60	**+6.0**
	Thpack8	-0.5
	Thpack9	**+0.7**
	BandR	-2.9

Example of an evolved programs

- (- (+ (- (+ (* NBH (- WW))(-(- (+ (* NBH (* (- W W) (- W W))) (- ((* (+ BH BWL) (- W W)) NBH) (* BH BH))) (- (+ (% H OBH) (+ W NBH)) (- (* W W) (* BH BH)))) (+ (* BHBH)(-(+BHBWL)OBH)))) (- (+ (* BH BH) (- (+ BH BWL) OBH)) (- W (* BH BH)))) (- (* (+ BHBWL)(- WW))(+BHBWL))) (- (+ (+ W BH) (+ W NBH)) (- (* WW)(* WW))))

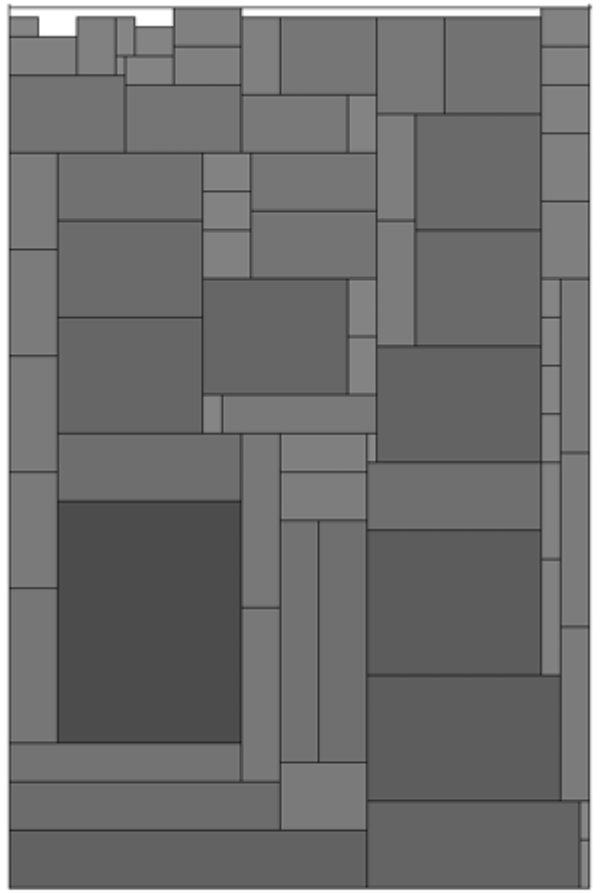


FIGURE 6.13: Packing of instance c5p3 to a height of 91. Packed by the heuristic evolved with population 64

Timetabling Practical

Workshop Exercise Generating and Applying GPs for Timetabling.

See Moodle to download the graph and complete the exercise – collaborate with your peers via the lecture discussion forum.

Concluding remarks

- **COMP2001 Reminders:**
  - Project coursework is on Moodle.
  - Remaining labs will be on HyFlex, Selection Hyper-heuristics, and used for project support.

- **Remaining Lectures:**
  - L9: Guest lectures on Fuzzy Logic and Symbolic AI.
  - [[BREAK]]
  - L10: Guest lecture [TBD]
  - L11: Revision for the exam

- **Module activities:**
  - Formative self-assessment quiz on Moodle.
  - **Next lecture with Ender**: 24th March 4pm.

University of Nottingham
UK | CHINA | MALAYSIA

Thank you

Any questions?